

A Datasets

A.1 The PAIRWISE dataset

Graph structure and node features. The PAIRWISE dataset consists of graphs that are simple chains of fixed length $L = 20$. We represent the chain as an undirected path on nodes $\{1, \dots, L\}$ with edges $(i, i + 1)$ for $i = 1, \dots, L - 1$, encoded as two directed edges $(i, i + 1)$ and $(i + 1, i)$; in our experiments we do *not* add self-loops. Each node i carries a k -dimensional feature vector $x_i \in \{0, 1\}^k$, where k is the number of categories (we denote the corresponding dataset by PAIRWISE_k). Node features are sampled independently as follows: first draw an activation flag $b_i \sim \text{Bernoulli}(p)$ with $p = 0.9$; if $b_i = 1$, sample a category $c_i \sim \text{Unif}\{1, \dots, k\}$ and set $(x_i)_{c_i} = 1$ and all other entries to 0, while if $b_i = 0$ we set $x_i = \mathbf{0}$. Thus each node is either all-zero or one-hot, with a high expected fraction p of active nodes. All random choices are made with a fixed seed (42) for reproducibility.

Label definition, deduplication, and split. Given a feature matrix $X \in \{0, 1\}^{L \times k}$ on the chain, the multi-label target $y \in \{0, 1\}^k$ is defined coordinate-wise by

$$y_c = \mathbf{1}\left\{\exists (i, j) \in E \text{ with } (x_i)_c = (x_j)_c = 1\right\}, \quad c = 1, \dots, k,$$

where E is the set of chain edges (self-loops, if present, are ignored when computing y). In words, class c is active if and only if there exists at least one edge whose two endpoints both carry the one-hot category c . To build the dataset, we repeatedly sample node features X as above, compute y , and retain only unique graphs, where uniqueness is defined by the node-feature pattern: we hash X (after casting to `uint8`) and reject duplicates until we have $n_{\text{train}} + n_{\text{test}} = 3000$ distinct samples. We then randomly shuffle these 3000 graphs (using the same seed 42) and split them into $n_{\text{train}} = 2000$ training and $n_{\text{test}} = 1000$ test graphs, corresponding to a test ratio of $1/3$.

A.2 The CONJUNCTION dataset

Data generation. We instantiate the CONJUNCTION dataset by drawing $n_{\text{train}} = 3000$ and $n_{\text{test}} = 1000$ graphs with a fixed random seed of 42. For each graph we first sample a latent category $c \in \{\text{none}, A\text{-only}, B\text{-only}, \text{both}\}$ from a categorical prior $(0.25, 0.25, 0.25, 0.25)$. Conditioned on c , we specify the *required* counts $r_\ell \in \{0, 1\}$ of cycles of lengths $\ell \in \{3, 4, 5, 6\}$ as follows. For A -only we set $(r_3, r_4, r_5, r_6) = (1, 0, 0, 1)$; for B -only $(0, 1, 1, 0)$; for both $(1, 1, 1, 1)$. For the “none” category we sample uniformly from the four non-target pairs $(1, 0, 1, 0)$, $(0, 1, 0, 1)$, $(1, 1, 0, 0)$, $(0, 0, 1, 1)$, which guarantees that neither conjunction $C_3 \wedge C_6$ nor $C_4 \wedge C_5$ holds. Given these required counts, we then draw independent extra copies $e_\ell \sim \text{Binomial}(8, 0.5)$ for each length ℓ with $r_\ell = 1$, and set the total motif counts to $n_\ell = r_\ell + e_\ell$; lengths with $r_\ell = 0$ remain absent. This corresponds to a symmetric Bernoulli prior over up to eight additional copies per present motif. The final labels are then computed deterministically from the counts as

$$y_A(G) = \mathbf{1}\{n_3 \geq 1 \wedge n_6 \geq 1\}, \quad y_B(G) = \mathbf{1}\{n_4 \geq 1 \wedge n_5 \geq 1\},$$

so that the four categories correspond to $(y_A, y_B) \in \{(0, 0), (1, 0), (0, 1), (1, 1)\}$ as intended.

Graph construction. Given the counts (n_3, n_4, n_5, n_6) , we materialize a concrete graph by placing all motif instances on disjoint node sets. For each of the n_ℓ copies of a cycle of length ℓ , we create a simple cycle on ℓ new nodes and designate one “anchor” node on that cycle. To increase local structural variety while preserving the core motifs, we attach “whisker” paths to each anchor: for every anchor we sample a number of whiskers $W \sim \text{Unif}\{0, 1, 2\}$, and for each whisker we sample a length $L_{\text{wh}} \sim \text{Unif}\{1, 2\}$ and attach a simple path of length L_{wh} starting from the anchor. Letting $\mathcal{A}$ denote the set of anchors of all motif instances, we then randomly permute $\mathcal{A}$ and connect consecutive anchors by simple paths of length $L_{\text{br}} \sim \text{Unif}\{1, 2\}$, ensuring that the overall graph is connected while the cycles themselves remain node-disjoint. Node features are one-dimensional and equal to the node degree (i.e., we use purely structural degree features with no learned or random attributes).

Deduplication and split. To avoid distributional artefacts from re-using the same graph topology, we perform bucketed rejection sampling with Weisfeiler–Lehman (WL) deduplication. We maintain four buckets (one per category) and generate candidate graphs as above until each bucket contains the desired number of graphs implied by the class prior (subject to rounding), rejecting any candidate

whose 1-WL hash (obtained from three iterations of 1-WL colour refinement on the undirected graph and hashing the resulting colour histogram) has already been seen in that bucket, with a cap of 60,000 proposals per bucket. Once a total of 4000 unique graphs have been collected, we perform a per-bucket random split into 3000 training and 1000 test graphs using the same seed (42), which preserves the class proportions up to integer rounding.

B Optimizer and training

B.1 Optimizer details

All models are trained with the Adam optimizer as implemented in PyTorch, using the default parameters $(\beta_1, \beta_2) = (0.9, 0.999)$ and $\epsilon = 10^{-8}$. Unless otherwise stated, we use a mini-batch size of 256, no learning-rate scheduling, and no early stopping; every configuration is run over multiple random seeds, but the optimizer hyperparameters are identical across seeds. No dropout is performed.

Table 2: Training hyperparameters (optimizer, learning rate, weight decay, and batch size) by experiment and architecture.

Experiment	Task / dataset	Model	Learning rate	Weight decay	Epochs
1	PAIRWISE ₁₆	GCN	0.10	0	300
1	PAIRWISE ₁₆	GIN	0.01	0	150
1	PAIRWISE ₁₆	GAT	0.01	0	150
2	CONJUNCTION	GCN	0.001	10^{-5}	1600
2	CONJUNCTION	GIN	0.001	10^{-5}	1600
2	CONJUNCTION	GAT	0.001	10^{-5}	1600
3	PAIRWISE ₁₆	GCN	0.10	0	200
3	PAIRWISE ₁₆	GIN	0.01	0	200
3	PAIRWISE ₁₆	GAT	0.01	0	200
4	PAIRWISE ₁₆	GCN	0.10	0	250
4	PAIRWISE ₁₆	GIN	0.01	0	250
4	PAIRWISE ₁₆	GAT	0.01	0	250

Experiment 1 (Dimension induced bottleneck on PAIRWISE). We use mean global pooling, batch size 256, and the learning rates and weight decays from Table 2. We train GCN models for 300 epochs and GIN/GAT models for 150 epochs:

GCN: lr = 0.10, wd = 0, 300 epochs; GIN/GAT: lr = 0.01, wd = 0, 150 epochs.

The optimizer is Adam with the above defaults and no scheduler.

Experiment 2 (Topology induced superposition). For the second experiment we again use Adam, batch size 256, and mean pooling, but employ smaller learning rates and a small amount of weight decay. All architectures share the same training schedule:

GCN/GIN/GAT: lr = 10^{-3} , wd = 10^{-5} , 1600 epochs.

Training is performed by calling `model.fit(train_loader, optimizer, num_epochs=1600)` with no additional regularization beyond weight decay.

Experiment 3 (alignment on PAIRWISE₁₆). In the alignment experiments on the PAIRWISE₁₆ dataset we sweep the hidden dimension over [10, 16, 22] with batch size 256, and the same Adam learning rates and weight decays as in Experiment 1 (see Table 2). We sweep the generalized-mean pooling exponent over

$$p \in \{1.0, 2.0, 4.0, 8.0, p_{\max}\},$$

where $p_{\max}$ denotes the largest exponent used in our code (approximating max pooling); all other optimization hyperparameters are kept fixed across p .

Experiment 4 (rank / selectivity on PAIRWISE₁₆). We use the ExperimentRank wrapper with hidden dimension 16, batch size 256, and the Adam settings from Table 2. Each model is trained via `model.fit(train_loader, optimizer, num_epochs=num_epochs)`, where `num_epochs` is a fixed constant within this experiment and does not depend on the random seed; no learning-rate schedule or additional regularization is applied beyond the specified weight decay.

C Generalized Mean Pooling

Let N be the number of nodes, $x_{id} \in \mathbb{R}$ the d -th dimension of node i , and $p \in \mathbb{R}$ a pooling parameter. Introduce a small stabiliser $\varepsilon > 0$ and define

$$g_{p,\varepsilon}(x) = \operatorname{sgn}(x) (|x| + \varepsilon)^p.$$

1. Element-wise transformation: For every node i and feature dimension d ,

$$\tilde{x}_{id} = g_{p,\varepsilon}(x_{id}) = \operatorname{sgn}(x_{id}) (|x_{id}| + \varepsilon)^p.$$

2. Pooling: Sum the transformed features and take their mean,

$$s_d = \sum_{i=1}^N \tilde{x}_{id}, \quad \bar{s}_d = \frac{1}{N} s_d.$$

3. Inverse transformation: Apply the signed p -th root to obtain the pooled feature

$$h_d = \operatorname{sgn}(\bar{s}_d) (|\bar{s}_d| + \varepsilon)^{1/p}.$$

The resulting graph-level representation is the vector

$$\mathbf{h} = (h_1, h_2, \dots, h_D).$$

Remark. Setting $p = 1$ recovers ordinary mean pooling (modulo the stabilisation term ε), while $p \rightarrow \infty$ recovers max pooling.

D Symmetry Breaking

Recall the stabilized generalized mean pooling from App. C. For each feature dimension d , let $x_{id} \in \mathbb{R}$ be the feature of node i , and define

$$\tilde{x}_{id} = g_{p,\varepsilon}(x_{id}) = \text{sgn}(x_{id})(|x_{id}| + \varepsilon)^p, \quad s_d = \sum_{i=1}^N \tilde{x}_{id}, \quad \bar{s}_d = \frac{1}{N} s_d.$$

The pooled feature is then

$$h_d = \text{sgn}(\bar{s}_d)(|\bar{s}_d| + \varepsilon)^{1/p}.$$

We now compute $\partial h_d / \partial x_{id}$, away from the measure-zero points where $x_{id} = 0$ or $\bar{s}_d = 0$ (so that the sign functions are locally constant).

Step 1: derivative of the inner transform. For fixed d and node i , write

$$\tilde{x}_{id} = \text{sgn}(x_{id})(|x_{id}| + \varepsilon)^p.$$

Using $\frac{\partial |x_{id}|}{\partial x_{id}} = \text{sgn}(x_{id})$ and $\text{sgn}(x_{id})^2 = 1$, we obtain

$$\frac{\partial \tilde{x}_{id}}{\partial x_{id}} = \text{sgn}(x_{id}) \cdot p(|x_{id}| + \varepsilon)^{p-1} \cdot \frac{\partial |x_{id}|}{\partial x_{id}} = p(|x_{id}| + \varepsilon)^{p-1}.$$

Hence

$$\frac{\partial s_d}{\partial x_{id}} = \frac{\partial}{\partial x_{id}} \left(\sum_{j=1}^N \tilde{x}_{jd} \right) = \frac{\partial \tilde{x}_{id}}{\partial x_{id}} = p(|x_{id}| + \varepsilon)^{p-1},$$

and therefore

$$\frac{\partial \bar{s}_d}{\partial x_{id}} = \frac{1}{N} \frac{\partial s_d}{\partial x_{id}} = \frac{p}{N} (|x_{id}| + \varepsilon)^{p-1}.$$

Step 2: derivative of the outer map. Define $y = \bar{s}_d$ and write

$$h_d = f(y) = \text{sgn}(y)(|y| + \varepsilon)^{1/p}.$$

For $y \neq 0$, $\text{sgn}(y)$ is locally constant, and we have

$$\frac{\partial h_d}{\partial y} = \text{sgn}(y) \cdot \frac{1}{p} (|y| + \varepsilon)^{1/p-1} \cdot \frac{\partial |y|}{\partial y} = \frac{1}{p} (|y| + \varepsilon)^{1/p-1},$$

since $\frac{\partial |y|}{\partial y} = \text{sgn}(y)$ and the sign factors cancel.

Step 3: chain rule. Combining the two steps by the chain rule,

$$\frac{\partial h_d}{\partial x_{id}} = \frac{\partial h_d}{\partial \bar{s}_d} \cdot \frac{\partial \bar{s}_d}{\partial x_{id}} = \frac{1}{p} (|\bar{s}_d| + \varepsilon)^{1/p-1} \cdot \frac{p}{N} (|x_{id}| + \varepsilon)^{p-1}.$$

Thus the stabilized gradient of generalized mean pooling is

$$\boxed{\frac{\partial h_d}{\partial x_{id}} = \frac{1}{N} (|\bar{s}_d| + \varepsilon)^{\frac{1}{p}-1} (|x_{id}| + \varepsilon)^{p-1}}$$

for all x_{id} and $\bar{s}_d$ away from the non-differentiable points of the absolute-value function.

Remark. If we further let $\varepsilon \rightarrow 0$ and assume non-negative activations so that $\bar{s}_d = s_d/N$ and $x_{id} \geq 0$, this reduces to $\frac{\partial h_d}{\partial x_{id}} = N^{-1/p} |s_d|^{1/p-1} x_{id}^{p-1}$.

E Sharp SI transitions during transition

A training-time phase transition. On PAIRWISE we observe two training modes: (i) *full-rank* runs where $r_\tau(H)$ rises to d and $\text{EffRank}(H)$ tracks it; (ii) *low-rank* runs where $r_\tau(H)$ remains $< d$ for most of training. Across seeds, SI/WNO of the *centroids* exhibit a reproducible four-stage pattern:

1. **Feature birth (noisy).** Early on, new class directions appear; $\text{EffRank}(C)$ rises $\Rightarrow$ $\text{SI} \downarrow$. Packing inside the span is crude, so WNO typically *increases*.
2. **Compression (sharp transition).** A narrow span forms: smallest singular values of H drop or remain suppressed, PC1 energy rises, and $\text{EffRank}(C)$ contracts $\Rightarrow$ a *sharp SI spike*. This point almost always coincides with the onset of overfitting (test loss starts to rise).
3. **Repacking (smooth).** One or two small singular directions “wake up”, allowing repacking *within* a slightly larger span: $\text{WNO} \downarrow$ and $\text{SI} \downarrow$ smoothly.
4. **Convergence (flat).** Geometry stabilizes; train loss drifts down, test loss drifts up.

The sharpness of Stage 2 reflects a discrete change in $r_\tau(H)$ (entire channels becoming globally inactive or reactivated), whereas WNO is computed *after* PC1 removal and need not jump at the same epoch.

F Geometric Regimes for Anti-Aligned Embedding Directions

F.1 Problem Formulation

We first consider the case where the feature number n is small and no hyperplane has formed: Let $\{\mathbf{v}_i \in \mathbb{R}^d \mid \|\mathbf{v}_i\|_2 = 1, i = 1, \dots, n\}$ be the (column-normalized) embedding directions learned by the network’s last hidden layer, and let $\{\mathbf{w}_i\}_{i=1}^n$ denote the corresponding unit-length rows of the classifier weight matrix. The anti-alignment hypothesis we explored in Section 6 states that after training

$$\mathbf{v}_i \approx \mathbf{w}_i \quad \text{and} \quad \mathbf{v}_i^\top \mathbf{v}_j \leq 0 \quad (i \neq j), \quad (\text{A.1})$$

i.e. every embedding is (almost) identical to its class weight, and pairwise inner products are non-positive if possible. In other words, the network attempts to realise a set of n mutually obtuse vectors in $\mathbb{R}^d$. The feasibility of (A.1) depends on the relation between n and d , which can be separated into four distinct regimes.

F.2 Regime Analysis

1. **Over-complete case**, $n > 2d$. Via a simple counting argument, at least two vectors must lie in the same closed half-space; hence their inner product is > 0 , so one pair is necessarily acute.
2. **Intermediate regime**, $d + 1 < n < 2d$. There are at most $n = d + 1$ mutually obtuse vectors. In this regime some vectors are mutually orthogonal and some are mutually obtuse.
3. **Simplex threshold**, $n = d + 1$. The largest set of vectors that can satisfy $\mathbf{v}_i^\top \mathbf{v}_j < 0$ for all $i \neq j$ is the vertex set of a centred regular d -simplex. This is the first value of n that permits pairwise obtuse directions.
4. **Under-complete case**, $n \leq d$. One can choose any n orthogonal unit vectors (or the n vertices of a degenerate n -simplex in a $n - 1$ dimensional subspace).

F.3 Implication for Rank Collapse

Assume training has converged to a low-rank subspace $\mathcal{S} \subset \mathbb{R}^d$ with $\dim(\mathcal{S}) = r < d$ and all $\mathbf{v}_i \in \mathcal{S}$. Activating a new latent dimension $\mathbf{e}_\perp \perp \mathcal{S}$ perturbs one vector as $\mathbf{v}_k \mapsto \tilde{\mathbf{v}}_k = \sqrt{1 - \varepsilon^2} \mathbf{v}_k + \varepsilon \mathbf{e}_\perp$. Then, for $i \neq k$,

$$\tilde{\mathbf{v}}_k^\top \mathbf{v}_i = \sqrt{1 - \varepsilon^2} \mathbf{v}_k^\top \mathbf{v}_i \geq \mathbf{v}_k^\top \mathbf{v}_i, \quad (\text{A.3})$$

because $\mathbf{v}_k^\top \mathbf{v}_i \leq 0$ in the metastable state. Hence the angle decreases⁵ towards 90° , which reduces pairwise obtuseness. As discussed in Section 6 this is penalised by the BCE loss function. During back propagation gradients oppose the escape from the low-rank basin, effectively creating a metastable minima. If ε grows large enough performance gains may begin to outweigh the angle penalty, and the rank grows.

F.4 Rank Collapse with Hyperplane

So far we have assumed that

$$\mathbf{v}_i \approx \mathbf{w}_i,$$

yet when a separating hyperplane emerges, the learned embeddings $\{\mathbf{v}_i\}$ and the classifier weights $\{\mathbf{w}_i\}$ are no longer almost identical. Empirically, $\mathbf{v}_i$ are often acute rather than mutually obtuse, but the *cross-class* dot products stay strictly negative:

$$\mathbf{w}_j^\top \mathbf{v}_i < 0 \quad (i \neq j). \quad (\text{A.4})$$

⁵Equality holds only when $\mathbf{v}_k^\top \mathbf{v}_i = 0$, keeping the angle unchanged at 90° .

Perturbation into a fresh dimension. Using the same notation as before, we perturb one embedding as

$$\tilde{\mathbf{v}}_k = \sqrt{1 - \varepsilon^2} \mathbf{v}_k + \varepsilon \mathbf{e}_\perp.$$

For any $j \neq k$

$$\mathbf{w}_j^\top \tilde{\mathbf{v}}_k = \sqrt{1 - \varepsilon^2} \mathbf{w}_j^\top \mathbf{v}_k + \varepsilon \mathbf{w}_j^\top \mathbf{e}_\perp > \mathbf{w}_j^\top \mathbf{v}_k, \quad (\text{A.5})$$

since $\mathbf{w}_j^\top \mathbf{v}_k < 0$ by (A.4) while $\mathbf{w}_j^\top \mathbf{e}_\perp = 0$. Thus the cross-class dot product becomes *less* negative, which increases BCE loss. In short, even after the hyperplane forms, the network remains trapped in a low-rank metastable basin for the same reason as before: cross-class obtuseness ($\mathbf{w}_j^\top \mathbf{v}_i < 0$) resists escapes that would reduce the margin.

F.5 Leaky ReLU Rank

Figure 7 illustrates the evolution of the rank of pooled embeddings across 15 training runs of a GIN model on the PAIRWISE dataset. The left graph presents results for a standard GIN, while the right graph shows outcomes when the MLP layers in the GIN architecture use leaky ReLU as the activation function.

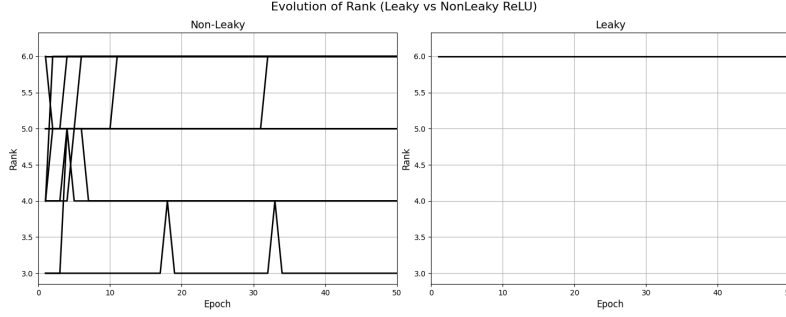


Figure 7: GIN model training runs with input dimension 12 and hidden dimensions [12, 6].